MS2 Summary · Regional Nectar-Flow Forecast

Elias Christen · I.BA_MLOPS HS26 · https://github.com/Whalesharkking/mlops · tag `ms2`

1. What I achieved in this milestone

- **What works now:** The feature pipeline runs on its own. Every morning a GitHub Actions job fetches the latest hive weights and the new weather forecast. It builds the features for the next ten days and saves them with the labels in my feature store on Hugging Face (`Whalesharkking/nectarflow-features`). If the forecast is not complete yet, the job waits for it. Every push also runs tests, a linter, a Docker build and a security scan.
- **Backfill:** The same code filled the history from 14.03.2024 to 17.10.2026. That is 940 days with ten rows each, done in 27 minutes. The dataset version is `2752f4c`.
- **Key decisions:**
  - Momentum only uses days up to $t_0 - 2$, because only those are final (see A1).
  - Each row stores the day of the forecast and the day it is for. So no future data can leak into training.
  - The data is versioned on the Hugging Face Hub instead of DVC. Every upload there is already a Git commit.
  - The pipeline never saves an incomplete weather forecast. On 15.10.2026 at 07:46 UTC, only 72 of 240 hours were available.
- **What it cost:**
  - The newest day is not used as an input.
  - The number of scales only exists from 17.10.2026 on. HiveWatch keeps no history of it.
  - Eight weather forecasts are missing in the Open-Meteo archive (05.-09.08.2025, 11.06., 23.06. and 19.09.2026). These days have no features.
  - 56 of 2813 days are flagged as implausible jumps, mostly snow in winter (see A2). A momentum window with such a day stays empty for up to 14 days.
  - The persistence baseline may now also only use final days. Recomputed on 2026, it reaches 0.31 at $h = 1$. Climatology reaches 0.29 at every horizon and is now the stronger baseline. The proposal had 0.24 and 0.32.
- **Look at first:** `.github/workflows/feature-daily.yml`, `src/nectarflow/features/pipeline.py` and `compute.py`, the dataset on Hugging Face, `tests/`.

2. Response to MS1 reviews

The labels come from the editorial summary. It covers every point of both reviews. The Must point comes first.

A4 · Permission to poll the bienen.ch endpoint · Partly

On 15.10.2026 I asked BienenSchweiz (cc HiveWatch) in writing for permission. There is no reply yet (last checked 17.10.2026). Until then the pipeline polls only once a day, names the source and publishes only regional values.

I tested the fallbacks on the same day:

- **Hiveeyes** has no scales in Switzerland. It is not usable.
- **midnightvalues** only covers 8 days and comes from the same provider. It is not independent.
- **TrachtNet Bayern** works. It is public, has daily corrected values per region and data from 2011. I asked for permission there too, also without reply.

TrachtNet stays a documented plan. I only build it if HiveWatch refuses or stops working. See the section Data sources in `README.md`.

A1 · Training/serving skew in the momentum features · Changed (`97c80d1`)

I confirmed the problem on 15.10.2026. At 09:00 UTC, the midnight value of the previous day was still missing in HiveWatch.

The suggested comparison over the backfill is not possible. The API only returns final values for the past. So I removed the skew by design:

- All momentum windows end at $t_0 - 2$. Training and daily runs see the same final days.
- Every run saves the last 30 days of the weight curve as it looked that day (`raw/hivewatch/`). These are the values "as known at t_0" that Reviewer A asked for. In MS3 they show how much recent days still change.

See `compute.py` (`momentum`), `labels.py` (`is_final`) and `pipeline.py`.

A2 · Changing scale population · Partly (`97c80d1`)

The set of scales does change. Today there are 364, but HiveWatch keeps no history. The effect on the label is small. HiveWatch averages weight changes, not absolute weights. A new or missing scale therefore hardly moves the average. A single faulty scale with +20 kg moves it by only 0.05 kg.

Bigger jumps come from weather on many scales at once, mainly snow. How I handle it:

- The number of scales is saved every day as `n_scales`.
- Days with an implausible change get a `jump_flag`. The limit is 2.5 kg from April to July and 1.0 kg otherwise.
- Flagged days are left out of the momentum and the climatology. The climatology uses the median.

56 of 2813 days are flagged. None is in the flow season, 21 are in January 2021 (snow). See `labels.py` and `quality/n_scales.parquet`.

A3 · Champion gate in winter · Not yet

I agree and will build it in MS3. One model forecasts all year. A new model can only become champion from March to September. It must also beat the current champion by at least 0.01 kg/day. That is about 3 % of the climatology error and below my 10 % goal. I will check this limit with the cross-validation from B2. From October to February, new models are trained and logged but never promoted.

B1 · Little training data for the later horizons · Not yet

I agree, and the backfill confirms it. Training has 644 days with 6,197 labelled rows, about 620 per horizon. 2,300 rows are in the flow season. The 2026 test has 2,805 rows. The limit is the weather archive, which starts on 14.03.2024. The hive data goes back to 2019.

In MS3 I will report the error per horizon together with these numbers. I will say honestly whether days 6 to 10 can beat climatology at all. A longer weather archive exists (GFS on AWS, from 2021), but it needs raw GRIB files. I only use it if the later horizons clearly lack data.

B2 · Time-ordered cross-validation for tuning · Not yet

I will build it in MS3 as suggested:

- The folds grow over time. All ten horizons of a day stay in the same fold.
- There is a 10-day gap before each validation block.
- The first validation block is the 2025 flow season.

The 2026 season stays untouched until the final test.